OPERATING SYSTEMS

MODULE 1 • REVISION NOTES + DIAGRAMS

Concept maps • process flows • real-life examples • interview points • self-test

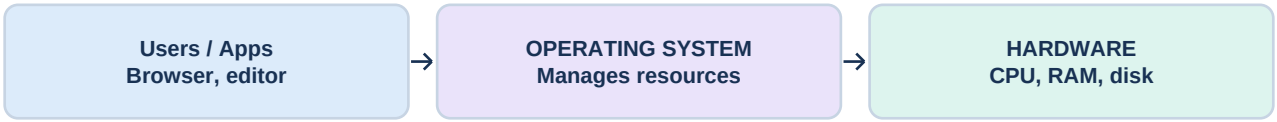
Topics included

Topic	Revision focus
1. Why do we need an OS?	OS as the bridge between applications/users and hardware.
2. OS as a manager	CPU, memory, files, devices, and access control.
3. Opening an app	Program → process → memory setup → CPU execution.
4. Apps running together	Multitasking, concurrency, parallelism, and scheduling.
5. OS and protection	Permissions, process isolation, user mode, kernel mode, system calls.

How to revise: Study each diagram first, explain it aloud in your own words, then try the self-test before checking the answer key.

1. Why Do We Need an Operating System?

An Operating System (OS) is system software that manages hardware resources and provides services to applications. It acts as a bridge between users/applications and computer hardware.



Main responsibilities

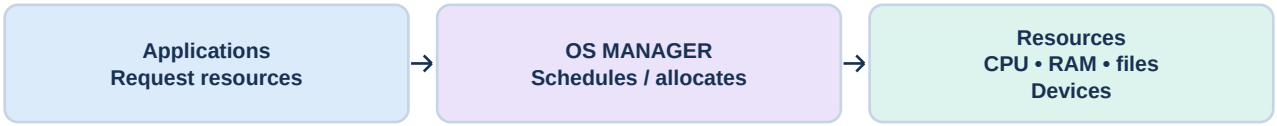
Function	What it does	Example
Process management	Runs and schedules tasks.	Switching between browser and music player.
Memory management	Allocates and tracks RAM.	Giving an app memory for working data.
File management	Organizes files and folders.	Saving a document.
Device management	Coordinates hardware and I/O.	Printing a document.
Protection / security	Controls access to resources.	Blocking unauthorized file changes.
User interface	Lets users interact with the computer.	Desktop, icons, terminal.

Memory tip: The OS makes hardware easier to use, shares resources, and helps keep programs from interfering with one another.

Interview answer: An OS is system software that manages hardware resources and provides common services and an execution environment for applications.

2. Operating System as a Manager

Think of the computer as a company: applications are workers requesting resources, hardware is shared equipment, and the OS is the manager coordinating access.



Resource	OS responsibility	Analogy
CPU	Chooses which ready task runs.	Assigning turns.
RAM	Allocates memory and tracks usage.	Assigning desk space.
Files	Organizes data and checks permissions.	Filing cabinet.
Devices	Coordinates input/output operations.	Managing a shared printer.
Access	Enforces permissions and isolation.	Restricted room access.

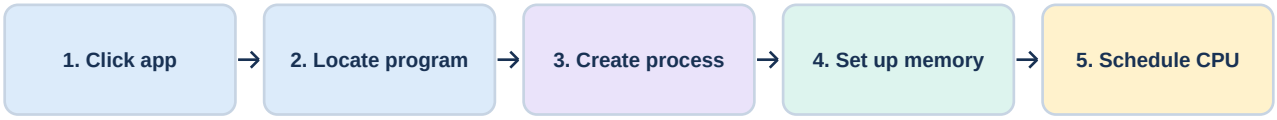
Why management matters

Several applications may request resources at once. The OS coordinates requests, reduces conflicts, keeps the system responsive, and enforces rules about access.

Memory trick: C-M-F-D-P
CPU • Memory • Files • Devices • Protection

3. What Happens When We Open an App?

A program stored on disk is not the same as a running process. When an app launches, the OS sets up the resources needed to execute it.



Step	What happens
1. Launch request	You click an icon or run a command.
2. Locate program	The OS identifies the executable and needed resources.
3. Create process	The OS creates a process and records information to manage it.
4. Set up memory	Code and data are loaded or mapped into the process's virtual address space.
5. Schedule execution	When ready, the process/thread can receive CPU time.
6. Initialize app	The application starts up and displays its interface; it may continue work in the background.

Program vs process

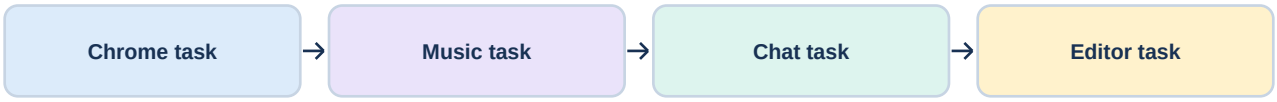
Program	Process
Passive set of instructions stored as a file.	Running instance of a program, with state and resources.
Example: browser executable on disk.	Example: browser currently running.

PCB (Process Control Block): A data structure used by the OS to store process-management information, such as process state and saved execution context. Exact fields vary by OS.

4. How Many Apps Run at the Same Time?

There is no fixed number. It depends on CPU cores, RAM, workload, and the OS. Many applications can be open, but the OS schedules runnable tasks.

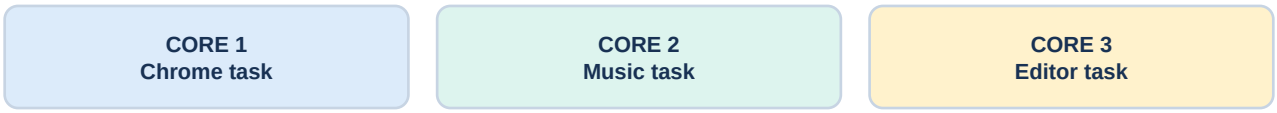
Single-core CPU: rapid switching



One core generally executes one thread at a time. The OS switches between runnable tasks quickly, so they appear to work together.

Concurrency: Multiple tasks make progress over the same period, even if they take turns on one core.

Multi-core CPU: parallel execution



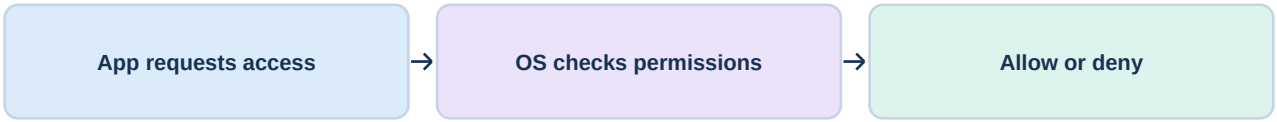
Different cores can execute instructions from different threads at the same instant. This is called parallelism.

Term	Meaning
Multitasking	The OS manages multiple tasks during the same period.
Concurrency	Tasks make progress over the same period; they may take turns.
Parallelism	Tasks execute at the same instant on separate processing units.
CPU scheduling	Selects a ready task/thread to receive CPU time.
Context switch	Saves one task's execution state and restores another's.

Memory trick: Single core → rapid switching can provide concurrency. Multiple cores → parallel execution is possible.

5. OS and Protection

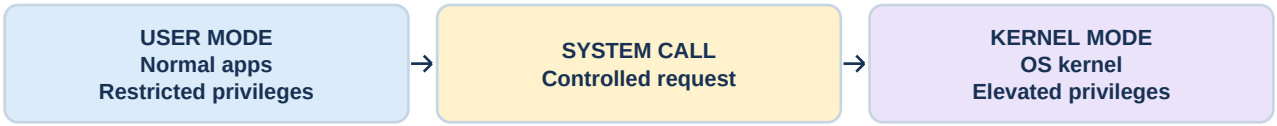
OS protection controls how users and processes access resources, restricting actions they are not authorized to perform.



What can be protected?

Resource	Example
Memory	A process should not freely overwrite another process's protected memory.
Files	A user may read a file but lack permission to modify it.
Devices	Access to cameras, disks, and other devices is controlled.
System operations	Privileged operations are restricted to authorized code/users.

User mode vs kernel mode



Applications typically run in user mode and request services through system calls. The kernel handles requests with elevated privileges and checks the relevant permissions. A system call is not automatic permission to do anything: the OS can reject a request.

Protection vs security

Protection	Security
Focuses on controlling access to resources.	Broader defense against threats and unauthorized access.
Example: file permissions and process isolation.	Example: authentication, encryption, updates, malware defenses.

Memory trick: Protection asks “Who can access what?” Security asks “How do we defend the system more broadly?”

6. Rapid Revision Sheet

Question / term	Short answer
What is an OS?	System software that manages hardware and provides services to apps.
Why need an OS?	Manage resources, run programs, organize files/devices, and control access.
OS as manager?	Coordinates CPU, memory, files, devices, and protection.
Program vs process?	Program = passive executable; process = running instance.
App launch flow?	Click → locate program → create process → set up memory → schedule CPU → initialize app.
Concurrency?	Tasks make progress over the same period, possibly by taking turns.
Parallelism?	Tasks execute at the same instant on different processing units.
Context switch?	Save one task's state and restore another's.
User vs kernel mode?	Restricted application execution vs elevated kernel privileges.
System call?	Controlled request from an application for an OS service.

7. Self-Test — Try Before Checking Answers

1. Name four major responsibilities of an OS.

2. Explain program vs process.

3. What allows a single-core CPU to share time among runnable tasks?

4. Explain concurrency vs parallelism.

5. What is a context switch?

6. In which mode do normal applications usually run?

7. What does a system call do?

8. Why might a protected file modification be denied?

Revision technique: Close the PDF and redraw each flow from memory. Explain what each arrow means, then give one practical example.

8. Self-Test Answer Key

Q	Expected answer
1	Any four: process/CPU management, memory management, file management, device management, user interface, protection/security.
2	A program is a passive set of instructions stored on disk; a process is an instance executing with state and resources.
3	CPU scheduling/time-sharing; the OS switches between runnable tasks.
4	Concurrency: tasks make progress over the same period, possibly by taking turns. Parallelism: tasks execute at the same instant on separate cores/processing units.
5	Saving one task's execution state and restoring another's when switching.
6	User mode, typically.
7	It lets an application request a controlled OS service, such as reading a file or creating a process.
8	The user or process may not have the required permission or privilege.

Final 30-second recap
The OS is the resource manager between applications and hardware. It sets up processes when apps launch, shares CPU time through scheduling, manages memory and devices, and protects resources using permissions, isolation, and privilege levels.

End of Module 1 revision notes